

Aufgabe 1: Schmucknachrichten

Teilnahme-ID: 76626

Bearbeiter/-in dieser Aufgabe:
Quirin Klemm

22. April 2025

Inhaltsverzeichnis

1	Lösungsidee	2
1.1	Perlen besitzen gleichen Durchmesser	2
1.2	Perlen mit unterschiedlichen Durchmesser	4
2	Umsetzung	5
2.1	Erstellung des Baums	5
2.2	Verbesserung des Baums	6
3	Laufzeit und Diskussion	7
3.1	Analyse des Algorithmus	7
3.2	Stand der Forschung	7
3.3	Fazit	8
4	Beispiele	8
5	Quellcode	8
6	Quellen	8

1 Lösungsidee

Eine Nachricht besteht aus mehreren unterschiedlichen Buchstaben. Für jeden Buchstaben ist ein Codewort zu finden, wobei insgesamt ein präfixfreier Code entstehen muss. Dabei soll die Gesamtlänge der codierten Nachricht möglichst gering sein. Daher wird ein präfixfreier Code gesucht, bei dem häufig vorkommende Buchstaben kurze Codewörter erhalten, während seltenere Buchstaben längere Codewörter zugewiesen bekommen. Im Allgemeinen gilt, solange alle Perlen im Durchmesser gleich sind:

$$c_i = p_i \cdot n_i \quad (1)$$

$$l = \sum_i c_i \quad (2)$$

wobei:

- c_i die Kosten des Buchstaben i ist,
- p_i die Häufigkeit des Buchstaben i in der Nachricht darstellt,
- n_i die Länge des Codewortes für den Buchstaben i ist,
- l die Länge der codierten Nachricht ist.

Wenn sich die Durchmesser der Perlen unterscheiden, stimmt diese Gleichung nicht mehr und muss anders berechnet werden.

Um präfixfreien Code zu erzeugen, wird am besten ein Baum benutzt. Da man durch diesen sicher gehen kann, dass keine Codewort der Präfix eines anderen Codewort ist. Dabei ist jedes Blatt des Baums ein Codewort und jede Kante zu dem Blatt eine Code vom Codewort. Der Baum hat mindestens so viele Blätter wie es verschiedene Buchstaben gibt. Jeder interne Knoten hat dabei maximal so viele Kinder, wie es verschiedene Perlen gibt. Die Ebene des Blattes entspricht die Länge des Codewortes.

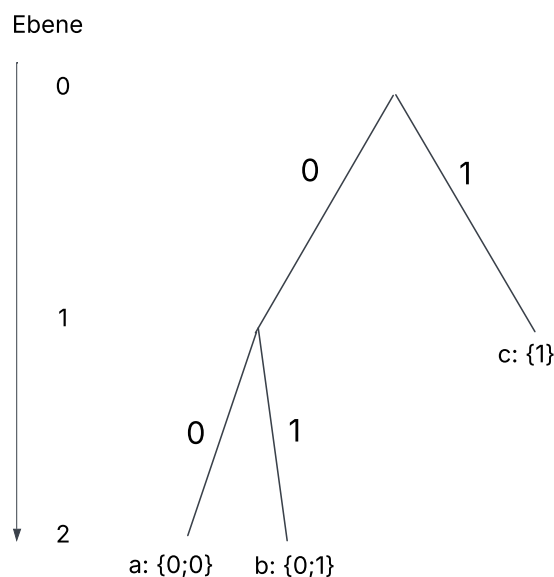


Abbildung 1: simple tree

1.1 Perlen besitzen gleichen Durchmesser

Um möglichst kleinen präfixfreien Code zu erstellen, muss der Buchstaben der am öftesten vorkommt ganz oben eingeordnet werden und die nächsten Buchstaben dann in absteigender Reihenfolge. In Abbildung 1 hätte c die höchste Häufigkeit und dannach kommt a und b . Was von a und b die höhere Häufigkeit hat, ist egal, da sie beide auf der untersten Ebene sind.

Daher ergibt sich $p_1 \geq p_2 \geq p_3 \dots \geq p_i$ und $n_1 \leq n_2 \leq n_3 \dots \leq n_i$

Zu finden ist also ein Baum, der basieren auf den Häufigkeiten der Buchstaben, seine Blätter so auf seine

Ebene so verteilt, dass die Gleichung (2) möglichst gering ist. Jeder interne Knoten muss dabei mindestens zwei Kinder haben, da ein interner Knoten mit nur einem Kind die Äste des Baums verlängert ohne irgendwelche anderen Vorteile dem Baum zu bringen.

Zu Beginn des Algorithmus wird ein perfekt ausbalancierter Baum erstellt, der so viele Blätter wie Buchstaben hat (Abbildung 2). Jedes Blatt wird von links nach rechts in aufsteigender Reihenfolge mit einem Buchstaben belegt, und die Gesamtkosten des Baums werden gemäß der Formel (1) berechnet. Dabei ergeben sich deutlich höhere Kosten für die rechten Blätter des Baums im Vergleich zu den linken, da alle Blätter auf derselben Ebene sind, jedoch die Häufigkeit der Buchstaben auf der rechten Seite größer ist. Um das auszugleichen, wird das Blatt mit den höchsten Kosten eine Ebene nach oben verschoben. Dies geschieht indem der Elternknoten des Blattes selber zum Blatt wird und alle Kinder verliert. Dadurch hat der Baum insgesamt zu wenig Blätter, weshalb er erweitert werden muss. Dies geschieht indem, alle möglichen neuen Blätter angeschaut werden und das mit den geringsten Kosten hinzugefügt wird. (Abbildung 3)

Das nach oben verschobenene Blatt erhält den Zusatz *protected*. Das verhindert, dass es selbst weitere Kinder bekommen kann. Dadurch wird ein Zyklus vermieden, bei dem ein Blatt zunächst nach oben verschoben und anschließend wieder nach unten erweitert würde. Das nach oben Verschieben und unten Neueinfügen der Blätter gilt als ein Optimierungsschritt, welcher im Optimierungsprozess wiederholt wird. (Abbildung 4) Dieser Vorgang wird so lange fortgesetzt, bis keine Blätter mehr verschoben werden können (Abbildung 5) - also keine weiteren Optimierungen mehr möglich sind. Am Ende wird der Baum mit den geringsten Gesamtkosten ausgegeben.

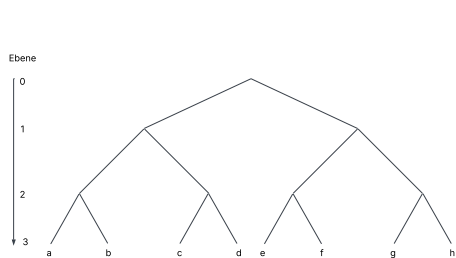


Abbildung 2: Balancierter Baum

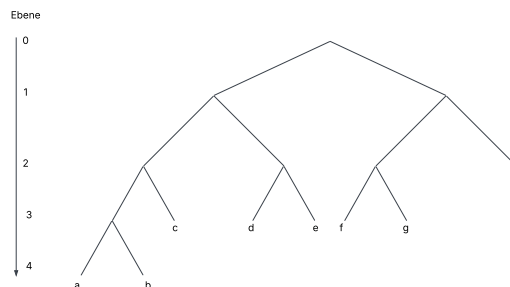


Abbildung 3: Transformation 1

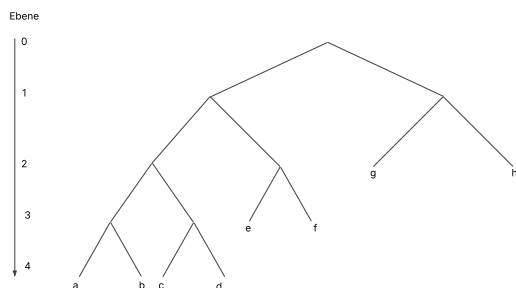


Abbildung 4: Transformation 2

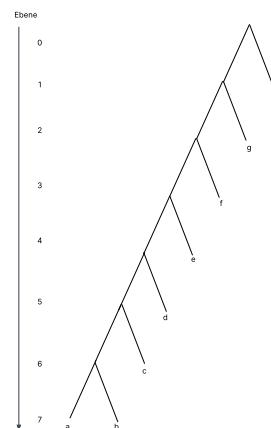


Abbildung 5: Ausgearteter Baum

1.2 Perlen mit unterschiedlichen Durchmesser

Das Problem bei unterschiedlichen Durchmesser besteht darin, dass Blätter auf derselben Ebene unterschiedliche Kosten haben können. Daher stimmt die Gleichung (1) nicht mehr, und muss statt dessen berechnet

$$c_i = p_i \cdot \sum_n cp_n \quad (3)$$

wobei:

- cp_n die Kosten/Durchmesser der Perle n ist.

Stellt man nun Bäume nicht nach ihrer Ebene, sondern nach ihren Kosten dar, erhält man einen *lopsided tree* [1].

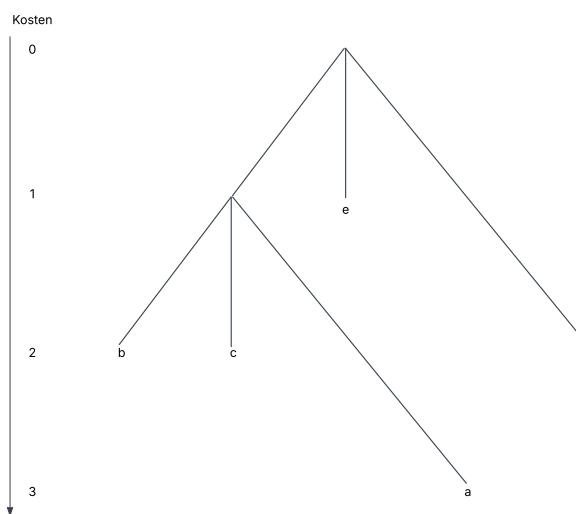


Abbildung 6: lopsided tree: Perlendurchmesser: (1; 1; 2)

Auch hier gilt: Jeder interne Knoten muss mindestens zwei Kinder haben. Bei dieser Art von Bäumen hat jedoch nicht der rechteste Knoten die geringsten Kosten, sondern der oberste. Die Kosten eines Knoten sind:

$$K = K_E + K_S \quad (4)$$

wobei:

- K_E die Kosten seines Eltern Knoten sind
- K_S seine eigenen, neuen Kosten sind

Der Algorithmus bleibt unverändert: Zunächst wird ein Baum erstellt, in dem alle Blätter ungefähr die gleichen Kosten haben. Anschließend wird der Buchstabe mit den höchsten Kosten nach oben verschoben, während der Baum unten erweitert wird. Nach jedem Schritt werden die gesamte Kosten des Baums berechnet (Gleichung (2)) und mit den bisherigen besten verglichen.

Wenn die Durchmesser alle gleich groß sind, gilt:

$$p_i \cdot n_i = p_i \cdot \sum_n cp_n \quad (5)$$

daher kann der zweite Algorithmus sowohl für gleiche als auch unterschiedliche Durchmesser benutzt werden.

2 Umsetzung

Die Lösung wurde in Python3 geschrieben. Die Eingabetexte werden eingelesen. Die Durchmesser der Perlen werden in einer Liste gespeichert und der Text in einem string. Aus dem Text werden anschließend zwei Listen erstellt: Die erste enthält die Häufigkeit der Buchstaben in absteigender Reihenfolge, die zweite listet die Buchstaben entsprechend dieser Sortierung auf.

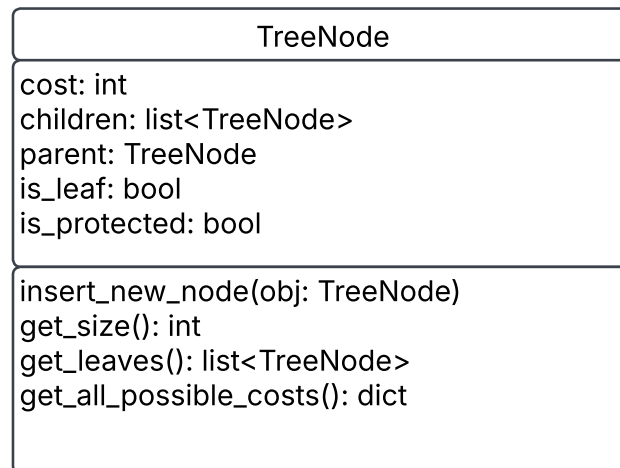


Abbildung 7: TreeNode Klasse

2.1 Erstellung des Baums

Zu Beginn wird der Wurzelknoten, root, mit einem balancierten Baum.

```
1 def create_tree(root: TreeNode) -> TreeNode:
```

Listing 1: Methode create_tree

Dabei werden jeweils alle potenziellen neuen Blätter im Baum betrachtet und das Blatt mit den geringsten Kosten wird dem Baum hinzugefügt. Falls der Elternknoten ein Blatt ist, entsprechen die Kosten des neuen Blatts den kombinierten Kosten für die ersten beiden Blätter. Das liegt daran, dass jeder interne Knoten mindestens zwei Kinder haben muss – daher stellen diese Kosten die effektiv neu entstehenden Kosten für den Baum dar. Das passiert solange, bis der Baum genau so viele Blätter hat, wie es Buchstaben gibt. Die möglichen neuen Blätter erhält man über den Aufruf

```
1 root.get_all_possible_costs()
```

```
1 def get_all_possible_costs(self) -> dict:
```

Listing 2: Methode get_all_possible_costs

Die Methode prüft, ob das TreeNode-Objekt noch weitere Kinder aufnehmen kann und nicht protected ist. Falls ja, fügt es sich selbst sowie die Kosten des potenziellen neuen Kindes einem Dictionary hinzu. Anschließend wird die Methode rekursiv für alle bestehenden Kinder aufgerufen, sodass auch deren mögliche Erweiterungen im Dictionary erfasst werden.

Neu Blätter werden über den Aufruf

```
1 root.insert_new_node(obj)
```

hinzugefügt

```
1 def insert_new_node(self, obj: TreeNode):
```

Listing 3: Methode `insert_new_node`

Die Methode prüft, ob das aktuelle Objekt selbst das gesuchte `TreeNode` Objekt ist. Falls ja, fügt es einen neues Kind hinzu. Wenn vorher noch keine Kinder vorhanden sind, werden zwei Kinder hinzugefügt. Ansonsten wird die Methode rekursiv auf alle bestehenden Kinder angewandt.

Über den Aufruf

```
1 root.get_size()
```

wird die Anzahl der Blätter ermittelt

```
1 def get_size(self) -> int:
```

Listing 4: Methode `get_size`

Die Methode gibt 1 zurück falls das Objekt ein Blatt ist. Ansonsten wird die Methode rekursiv bei den Kinder aufgerufen und die Summe zurückgegeben

2.2 Verbesserung des Baums

Sobald ein balancierter Baum erstellt wurde, muss dieser optimiert werden. Dies geschieht über die Methode

```
1 def find_best_tree(root: TreeNode) -> TreeNode:
```

Listing 5: Methode `find_best_tree`

Die Methode versucht, den übergebenen Baum so lang zu optimieren, bis alle Blätter als *protected* markiert sind. Dazu wird der Optimierungsschritt wiederholt und jeweils die neuen Kosten des Baums berechnet. Sind die neuen Kosten ein neues Minimum, wird der aktuelle Baum als bester Zwischenstand gespeichert. Kann der Baum nicht weiter verbessern werden und es tritt ein Fehler auf, wird der bis dahin beste gefundene Baum zurückgegeben.

Der Optimierungsschritt des Baums geschieht über die Methode

```
1 def improve_tree(root: TreeNode) -> TreeNode:
```

Listing 6: Methode `improve_tree`

Die Methode holt sich erstmal alle Blätter im Baum über den Aufruf

```
1 root.get_leaves()
```

Diese Liste wird dann aufsteigend sortiert und die Kosten für jedes Element können mit der Gleichung (3) berechnet werden. Der Elternknoten des Blatts mit den höchsten Kosten wird daraufhin selbst zu einem Blatt. Dafür werden die Attribute vom Elternknoten

```
1 is_leaf = True
  is_protected = True
3 children = []
```

gesetzt. Um die fehlenden Blätter wieder hinzuzufügen wird noch die Methode 1 aufgerufen.

Die gesamt Kosten des Baums wird durch die Methode

```
1 def calculate_tree(root: TreeNode) -> TreeNode:
```

Listing 7: Methode `calculate_tree`

berechnet. Die Methode holt sich erstmal alle Blätter im Baum über den Aufruf

```
1 root.get_leaves()
```

Die Liste wird dann aufsteigend sortiert und jedes Element wie in der Gleichung (3) berechnet und zusammenaddiert.

3 Laufzeit und Diskussion

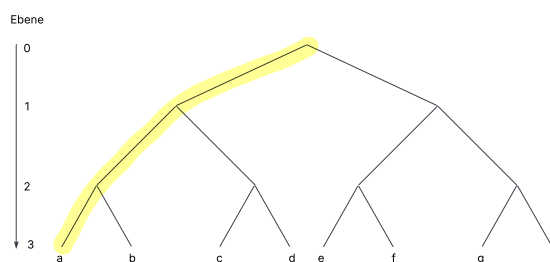


Abbildung 8: Balancierter Baum, richtige Knoten markiert

3.1 Analyse des Algorithmus

Der Algorithmus endet, wenn keine Knoten mehr verschoben werden können. Dies ist gegeben, wenn der Baum maximal links ausgeartet ist (Abbildung 5). Allgemein kann jeder Knoten maximal $n - 1$ Mal verschoben werden, bis er komplett auf der linken Seite ist. Dabei ist n die Anzahl an verschiedenen Perlen. Daraus folgt, dass es maximal

$$I < (n - 1) \cdot \text{Anzahl falscher Knoten} \quad (6)$$

Iterationen benötigt werden, bis zum maximal ausgearteten Baum. Falsche Knoten sind dabei innere Knoten, die beim ausbalancierten Baum noch nicht auf der richtigen Stelle sind, also alle bis auf die linken Knoten. (Abbildung 8)

Generell gilt um die maximale Anzahl an innere Knoten zu kriegen, muss jeder Knoten eine möglichst geringe Anzahl an Kindern haben.

Daher ergibt sich allgemein eine Laufzeit von $O(n^{l+1} - l) = O(n^{\log_n w + 1} - \log_n w)$.

3.2 Stand der Forschung

Die vorliegende Fragestellung ist seit den 60iger Jahren in der theoretischen Informatik bekannt. Dabei lieferte Huffman eine Lösung für den spezial Fall, von zwei Quellsymbolen.

Für unseren Fall von mehr als zwei Quellsymbolen mit unterschiedlichen Kosten, entwickelten Mordecai J. Golin und Günter Rote im Jahr 1998 das bis jetzt schnellste Verfahren, um auf eine exakte Lösung zu kommen.

"The minimum-cost prefix-free code for n words can be computed in $O(n^{C+2})$ time and $O(n^{C+1})$ space, if the costs of the letters are integers between 1 and C ."

Für die Beispielaufgabe *schmuck5.txt* hat der Algorithmus eine Laufzeit 34^8 ($n=34$; $C=6$) Iterationen. Dieser Art von Algorithmus kann wegen seiner Laufzeit nicht für die vorgegebenen Beispielaufgabe verwendet werden. [1]

Um eine polynominelle Laufzeit zu erreichen, entwickelten Mordecai J. Golin, Claire Mathieu und Neal E. Young 2012 ein approximations Verfahren.

Dieser Algorithmus passiert auf dem PTAS-Verfahren, wobei zuerst ein präfixfreier Code bis zur einer bestimmten Tiefe τ erstellt wird und dann die restlichen Codewörter durch ein Rundungsverfahren ergänzt werden. Dabei ist es zentral, dass die Wahrscheinlichkeitsverteilung der zu kodierenden Codewörter möglichst in große Gruppen mit gleich großer Wahrscheinlichkeit ein geordnet werden können. In den Beispielen, vorallem in Beispiel 9 ist dies nicht gegeben, weshalb das Bestimmen des Vorbaums über 9^{24} Schritte benötigt. Somit ist dieses Verfahren für die gewählten Beispiele ebenfalls nicht verwendbar. [2]

3.3 Fazit

Der vorgestellte Algorithmus stellt keine exakte Lösung für das gegebene Problem dar, liefert jedoch in polynomieller Laufzeit eine gute Approximation. Aus diesem Grund erfolgt durch das vorgestellte Verfahren keine Beantwortung, ob das Problem NP-hart ist. Im Gegensatz zum oben vorgestellten Verfahren kann der Algorithmus auch Fälle verarbeiten, in denen die Wahrscheinlichkeiten der Zeichen nicht gleichmäßig verteilt oder in klaren Gruppen strukturiert sind.

In praktischen Tests zeigte sich, dass die berechneten Lösungen im Durchschnitt nur etwa 10 % von der optimalen Lösung abweichen. Ein formaler Beweis dieser Beobachtung steht allerdings noch aus.

4 Beispiele

Genügend Beispiele einbinden! Die Beispiele von der BwInf-Webseite sollten hier diskutiert werden, aber auch eigene Beispiele sind sehr gut – besonders wenn sie Spezialfälle abdecken. Aber bitte nicht 30 Seiten Programmausgabe hier einfügen!

5 Quellcode

Unwichtige Teile des Programms sollen hier nicht abgedruckt werden. Dieser Teil sollte nicht mehr als 2–3 Seiten umfassen, maximal 10.

6 Quellen

[1] Mordecai J. Golin and Günther Rote, *A Dynamic Programming Algorithm for Constructing Optimal Prefix-Free Codes with Unequal Letter Costs*, IEEE Transactions on Information Theory 44, no. 5, (September 1998)

[2] Mordecai J. Golin, Claire Mathieu und Neal E. Young, *HUFFMAN CODING WITH LETTER COSTS: A LINEAR-TIME APPROXIMATION SCHEME*, 2012 Society for Industrial and Applied Mathematics Vol. 41, No. 3, pp. 684–713